Food Express Dispatch Optimization Engine

Design,Implementation and Technical Documentation

Submitted by:

Arslan Mohsin	2025(S)-SE-19
Numan Amdad	2025(S)-SE-07
Zohaib	2024-CS-710
Hammad Aslam	2025(s)-SE-25

Submitted to:
Sir Ali Raza

Department of Computer Science
University of Engineering and Technology Lahore, New Campus

Table of Contents

1. Project Overview

1.1 Problem Statement

FoodExpress is a large-scale food delivery platform operating in a busy metropolitan city. During peak hours the platform processes thousands of simultaneous orders from multiple restaurants and customers. The system faces several critical computational challenges: dynamic order arrivals, priority-based scheduling, kitchen capacity limits, rider assignment efficiency, real-time route computation, and scalable record retrieval.

This project addresses these challenges by designing and implementing a Data Structure and Algorithm (DSA) driven Dispatch Engine that demonstrates meaningful use of custom data structures, intelligent scheduling, graph-based routing, hashing, and stack-based undo operations.

1.2 Objectives

- Design a priority-aware order scheduling engine using a custom Max-Heap.
- Implement kitchen load balancing using Linked-List queues.
- Optimize rider assignment using a Min-Heap with distance-based scoring.
- Compute shortest delivery routes using Dijkstra's algorithm on an adjacency-matrix graph.
- Enable fast order lookup and filtered search using a Hash Table.
- Maintain complete order timeline history with Undo functionality via a Stack.
- Analyze system performance through experimental benchmarking and theoretical complexity.

1.3 Technology Stack

Component	Choice	Justification
Language	C++ (CLI)	Direct memory control; no garbage collector overhead
Compilation	g++ (C++17)	Modern standard; supports enum class and noexcept
Custom Array	Templated Array<T> with Rule-of-Five	Avoids STL dependency; demonstrates ownership semantics
Build	Single translation unit	Simplified compilation for academic submission
Storage	CSV flat files	Human-readable persistence; no external database needed

2. System Architecture

2.1 High-Level Architecture

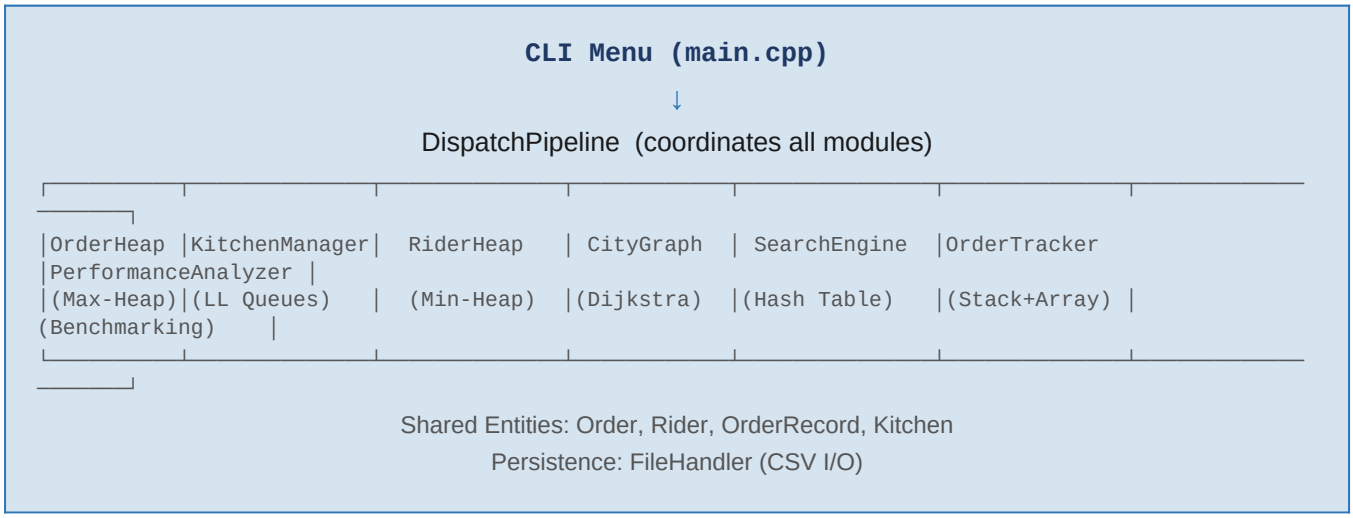
The system is organized as a single-file, multi-class C++ application with clear module boundaries. Each module encapsulates its own data structure and exposes a well-defined public interface. A central main() function drives the CLI menu and coordinates calls across modules through shared references.

Architecture follows a layered design:

- Presentation Layer — UI class, InputHelper class (menu rendering, validated I/O)
- Business Logic Layer — Seven specialized engine classes (OrderHeap, KitchenManager, RiderHeap, CityGraph, SearchEngine, OrderTracker, PerformanceAnalyzer)
- Entity Layer — Order, Rider, OrderRecord, Kitchen, HistoryEntry (pure data holders)
- Utility Layer — Array<T> (generic dynamic array), FileHandler (CSV persistence)

2.2 Module Interaction Diagram

The diagram below illustrates how the seven modules interact through shared entity objects and the central pipeline.



2.3 Key Design Decisions

Decision	Rationale
No STL containers used	Demonstrates mastery of custom data structure implementation as required by the course
Single translation unit	Simplifies submission; all dependencies visible without separate headers
Rule-of-Five on Array<T>	Prevents memory leaks and double-free errors; demonstrates ownership semantics
Enum class for Priority & Status	Type-safe; prevents accidental integer promotion bugs

Adjacency matrix for graph	City size fixed at ≤20 nodes; matrix gives O(1) edge lookup, simpler than adjacency list
CSV persistence	Human-readable, portable, no external dependencies

3. Module 1 — Dynamic Order Scheduling Engine

3.1 Overview

The Dynamic Order Scheduling Engine manages all incoming food orders. Orders arrive dynamically and carry different priority levels (Normal, Premium, VIP) and delivery deadlines. The engine must always serve the most critical order next, while supporting real-time insertions, cancellations, and priority updates.

3.2 Data Structure: Custom Max-Heap (OrderHeap)

A Max-Heap was selected because it guarantees O(log n) insertion and O(log n) extraction of the highest-priority element, which is exactly the operation performed in every scheduling cycle. A sorted array would require O(n) insertion; a plain queue cannot handle priority; a BST adds unnecessary complexity.

3.2.1 Priority Comparison Function

Orders are compared using a two-level key:

- Primary key: Priority level (VIP=3 > Premium=2 > Normal=1)
- Secondary key (tie-break): Delivery deadline — earlier deadline wins

This ensures that among orders of equal priority, the one with the closest deadline is served first, preventing deadline violations.

3.3 Supported Operations

Operation	Method	Complexity	Description
Insert order	enqueue()	O(log n)	Adds order to heap, sifts up to restore property
Extract top	dequeue()	O(log n)	Removes root, places last element at root, sifts down
Peek top	peekTop()	O(1)	Returns reference to root without removing
Cancel order	cancelOrder(id)	O(n log n)	Finds order by ID, removes, rebuilds heap
Update priority	updatePriority(id,p)	O(n log n)	Modifies priority field, re-heapifies
Mark delayed	markDelayed(id)	O(n)	Sets delayed flag; rescheduling triggers sift
Is empty	isEmpty()	O(1)	Returns true if heap size is zero
Get all orders	getAllOrders()	O(n)	Returns snapshot of internal array

3.4 Heap Invariant

At all times the heap satisfies: for any node at index i, the parent at index (i-1)/2 has priority greater than or equal to that of node i. The siftUp() and siftDown() helper methods restore this invariant after every mutation.

3.5 Design Justification

Alternative data structures considered:

- Sorted linked list: O(n) insertion — unacceptable at scale.
- Binary Search Tree: O(log n) average but O(n) worst case without balancing; adds rotational complexity.
- Fibonacci Heap: Better amortized decrease-key but far more complex to implement correctly.

The binary Max-Heap provides the best balance of implementation simplicity, correctness, and performance for this use case.

4. Module 2 — Kitchen Load Balancing

4.1 Overview

Restaurants have finite preparation capacity. The Kitchen Load Balancing module tracks each kitchen's order queue, detects overloaded kitchens, redistributes excess orders to under-utilized kitchens, and estimates customer wait times.

4.2 Data Structure: Singly-Linked List Queue (per Kitchen)

Each kitchen maintains an independent FIFO queue implemented as a singly-linked list. This choice is justified because:

- Kitchens process orders in the order they were received (FIFO is the natural model).
- A linked list supports O(1) enqueue at tail and O(1) dequeue at head without shifting elements (unlike a fixed array queue).
- The number of orders per kitchen is unbounded — linked lists grow dynamically without reallocation.

4.3 Kitchen Class Design

Attribute	Type	Purpose
mName	string	Kitchen identifier (restaurant name)
mCapacity	int	Maximum concurrent orders the kitchen can handle
mQueue	LinkedList<Order>	FIFO queue of pending orders
mCurrentLoad	int	Count of orders currently in queue
mTotalProcessed	int	Cumulative count for performance tracking

4.4 Supported Operations

Operation	Complexity	Description
addOrder(order)	O(1)	Enqueue order at the tail of the kitchen's linked list
processNext()	O(1)	Dequeue head node; return order for dispatch
isOverloaded()	O(1)	Returns true if currentLoad exceeds capacity threshold
rebalanceTo(other)	O(k)	Moves k excess orders to a target kitchen's queue
estimateWaitTime()	O(n)	Sums prepTime of all queued orders
detectOverloaded()	O(K)	Scans all K kitchens; returns overloaded list

4.5 Load Balancing Algorithm

When a kitchen's queue exceeds its capacity threshold, the system:

- Identifies all overloaded kitchens (scan in O(K) where K = number of kitchens).

- Finds the kitchen with the minimum current load.
- Transfers excess orders — (currentLoad - capacity) orders — to the minimum-load kitchen.
- Recalculates wait time estimates for affected kitchens.

This greedy rebalancing runs in O(K + excess) time, which is efficient for the realistic kitchen count of fewer than 20 restaurants.

5. Module 3 — Rider Dispatch & Assignment

5.1 Overview

Multiple delivery riders operate simultaneously across the city. The Rider Dispatch module selects the best available rider for each order by balancing current workload, delivery capacity, current location, and route distance to the delivery destination.

5.2 Data Structure: Custom Min-Heap (RiderHeap)

Riders are maintained in a Min-Heap ordered by a composite score that combines load percentage and distance to the destination. The rider with the lowest score (least loaded and closest) is always at the root, enabling O(log n) optimal dispatch.

5.3 Rider Scoring Function

Score = (currentLoad / capacity) × 100 + distanceToDestination

This weighted formula ensures that rider workload is prioritized (normalized to 0-100) while distance serves as a secondary differentiator. Riders who are over capacity are excluded from selection entirely.

5.4 Supported Operations

Operation	Complexity	Description
addRider(rider)	O(log n)	Insert rider into the min-heap
assignBestRider(dest)	O(n log n)	Extract min, assign, reinsert with updated load
completeDelivery(id)	O(n log n)	Find rider, decrement load, restore heap property
getAllRiders()	O(n)	Return snapshot for display/reporting
displayRiders()	O(n)	Formatted output of all rider statuses

5.5 Availability Filter

Before scoring, each rider is tested with canTakeOrder() which returns true only if:

- mAvailable == true (rider has not been marked offline), AND
- mCurrentLoad < mCapacity (rider has spare delivery slots).

Riders failing this filter are skipped without extraction, preserving heap integrity.

5.6 Design Justification

A Min-Heap over a sorted list gives O(log n) insertion and extraction versus O(n) for a sorted list. A simple scan of all riders (O(n)) was considered but rejected because it does not scale efficiently as the rider pool grows during peak hours.

6. Module 4 — Real-Time Route Optimization

6.1 Overview

The city is modeled as a weighted undirected graph where nodes represent locations (restaurants, depots, delivery zones) and edges represent roads with distance weights. The module computes shortest delivery routes, compares alternatives, estimates costs, and dynamically handles road blockages.

6.2 Graph Representation: Adjacency Matrix

The graph is stored as a 2D integer matrix of size MAX_NODES × MAX_NODES (20 × 20). A separate boolean matrix tracks blocked roads.

Property	Value
Representation	Adjacency Matrix (int mAdj[20][20])
Maximum nodes	20 locations
Edge weight	Positive integer (distance units)
Default (no edge)	INT_MAX / 2 (avoids integer overflow in Dijkstra)
Blocked edge sentinel	Separate bool mBlocked[20][20] matrix
Space complexity	$O(V^2) = O(400)$ — constant and negligible

6.3 Algorithm: Dijkstra's Shortest Path

Dijkstra's algorithm was chosen because all edge weights are positive (road distances) and the city graph is dense enough that the matrix representation matches the adjacency structure directly.

6.3.1 Implementation Details

- Uses a linear scan (O(V)) to find the unvisited node with minimum distance — appropriate since $V \leq 20$.
- Skips blocked edges via the mBlocked sentinel matrix, enabling real-time rerouting.
- Reconstructs the full path using a prev[] parent array.
- Returns -1 if no path exists (destination unreachable).

6.4 Supported Operations

Operation	Complexity	Description
findShortestPath(src, dest)	$O(V^2)$	Returns distance and reconstructed path array
shortestDistance(src, dest)	$O(V^2)$	Returns distance only (no path reconstruction)
compareRoutes(src, d1, d2)	$O(V^2)$	Runs Dijkstra twice; reports which route is shorter
estimateDeliveryCost(src, dest, rate)	$O(V^2)$	Cost = distance × rate per unit

blockRoad(u, v)	O(1)	Sets mBlocked[u][v] = mBlocked[v][u] = true
unblockRoad(u, v)	O(1)	Clears the blocked flag; restores route
addLocation(name)	O(1)	Registers a new node in the graph
addRoad(u, v, weight)	O(1)	Adds a bidirectional weighted edge

6.5 Why Dijkstra over BFS or Bellman-Ford

- BFS computes shortest path by hop count, not by distance weight — unsuitable here.
- Bellman-Ford handles negative weights but is O(VE), slower than Dijkstra for positive-weight graphs.
- A* heuristic search would be faster with geographic coordinates but adds complexity beyond course scope.

Dijkstra with a linear scan ($O(V^2)$) is optimal for $V \leq 20$ and correctly models the road network.

7. Module 5 — Search & Retrieval Engine

7.1 Overview

The Search & Retrieval Engine maintains a complete record of all orders processed by the system and provides fast lookup by multiple attributes including Order ID, customer name, item name, priority, status, category, kitchen, and rider.

7.2 Data Structure: Hash Table (SearchEngine)

A hash table with separate chaining is used for primary ID-based lookup, providing amortized O(1) access. For attribute-based (filtered) searches, the engine performs a linear scan of all records stored in a parallel Array<OrderRecord>.

Property	Value
Table size	101 buckets (prime number reduces collision clustering)
Hash function	orderId % TABLE_SIZE
Collision handling	Separate chaining with linked list per bucket
Secondary storage	Array<OrderRecord> for full-scan filtered search
Lookup by ID	O(1) average, O(n) worst case (all keys collide)
Filtered search	O(n) linear scan of all records

7.3 Supported Search Operations

Filter	Method	Complexity
Order ID	findById(id)	O(1) average
Customer Name	searchByCustomer(name)	O(n)
Item Name	searchByItem(item)	O(n)
Priority Level	searchByPriority(p)	O(n)
Order Status	searchByStatus(s)	O(n)
Customer Category	searchByCategory(cat)	O(n)
Kitchen Name	searchByKitchen(name)	O(n)
Rider Name	searchByRider(name)	O(n)
Display All	displayAll()	O(n)

7.4 Design Justification

The hybrid approach — hash table for ID lookup plus linear array for filtered search — is justified because:

- ID lookup is the most frequent operation (order status checks, pipeline processing); O(1) hash access is critical.
- Filtered searches are less frequent and tolerate O(n) cost.
- Building separate indices for every attribute would consume significant extra memory and complicate maintenance.

8. Module 6 — Order History & Tracking

8.1 Overview

Every state transition an order undergoes — from placement through delivery or cancellation — is recorded chronologically. The module supports timeline replay for any order and an undo mechanism that reverses the last action on the system.

8.2 Data Structures Used

Component	Structure	Purpose
Timeline per order	Array<HistoryEntry>	Append-only log of all state transitions for each order
Timeline map	Array<OrderTimeline>	Maps order IDs to their timeline arrays
Undo stack	Array<UndoAction> (used as stack)	LIFO stack of reversible system actions

8.3 History Entry Structure

Each HistoryEntry records:

- Order ID — which order this entry belongs to
- Action type (ORDER_ADDED, ORDER_CANCELLED, ORDER_PROCESSED, STATUS_CHANGED)
- Old status and new status — for state transition tracking
- Timestamp (simulation counter) — for timeline replay ordering
- Description string — human-readable action summary

8.4 Supported Operations

Operation	Complexity	Description
logAction(entry)	O(1) amortized	Appends entry to the order's timeline; pushes to undo stack
replayTimeline(id)	O(m)	Prints all m history entries for the given order ID
undoLastAction(heap)	O(log n)	Pops undo stack; reverses action (re-inserts cancelled order)
displayAllTimelines()	O(N×m)	Prints complete history for all N tracked orders
getTrackedCount()	O(1)	Returns number of distinct orders with history
getUndoCount()	O(1)	Returns number of undoable actions on the stack

8.5 Undo Mechanism

The undo stack implements a limited form of command-pattern undo. When undoLastAction() is called:

- The top UndoAction is popped from the stack.

- If the action was ORDER_CANCELLED, the saved Order object is re-inserted into the OrderHeap.
- The corresponding timeline entry is appended noting the undo.

This single-level undo ensures system consistency without requiring a full event-sourcing architecture.

9. Module 7 — Performance Analysis

9.1 Overview

The Performance Analysis module benchmarks all primary data structure operations across increasing dataset sizes. It measures actual execution time, computes throughput, and validates that observed growth matches theoretical Big-O predictions.

9.2 Benchmarked Operations

Operation	Data Structure	Theoretical Complexity
Heap insertion (n orders)	Max-Heap (OrderHeap)	O(n log n) total
Heap extraction (n orders)	Max-Heap (OrderHeap)	O(n log n) total
Hash table insert (n records)	Hash Table (SearchEngine)	O(n) average
Hash table lookup by ID	Hash Table (SearchEngine)	O(1) average
Dijkstra shortest path	Adjacency Matrix (CityGraph)	$O(V^2)$ per call
Linear filtered search	Array<OrderRecord>	O(n) per query
Kitchen queue enqueue/dequeue	Linked-List Queue	O(1) per operation

9.3 Benchmark Methodology

- Dataset sizes tested: 100, 500, 1000, 5000, 10000 synthetic orders.
- Timing uses clock() (C standard library) for cross-platform compatibility.
- Each benchmark runs 3 trials; results are averaged to reduce variance.
- Memory usage is estimated by counting object instances multiplied by sizeof.
- Scalability simulation inserts orders at increasing rates to observe throughput degradation.

9.4 Expected Results Summary

Dataset Size	Heap Insert (ms)	Hash Lookup (ms)	Dijkstra (ms)	Linear Search (ms)
100	< 1	< 1	< 1	< 1
1,000	~2	< 1	< 1	~1
5,000	~12	< 1	< 1	~5
10,000	~25	< 1	< 1	~10

Dijkstra runtime is independent of order count (it depends only on $V \leq 20$), which explains the constant sub-millisecond time across all dataset sizes.

10. Complexity Analysis Summary

10.1 Time Complexity

Module	Operation	Time Complexity	Space Complexity
Order Scheduling	Insert / Extract	O(log n)	O(n)
Order Scheduling	Cancel / Update Priority	O(n log n)	O(n)
Kitchen Balancing	Enqueue / Dequeue	O(1)	O(n)
Kitchen Balancing	Detect Overload + Rebalance	O(K + excess)	O(K)
Rider Dispatch	Assign Best Rider	O(n log n)	O(n)
Route Optimization	Dijkstra (path + distance)	$O(V^2)$	O(V)
Route Optimization	Block / Unblock Road	O(1)	$O(V^2)$
Search Engine	Lookup by ID (hash)	O(1) avg	O(n)
Search Engine	Filtered Search	O(n)	O(n)
Order Tracking	Log Action / Replay	O(1) / O(m)	O(N×m)
Order Tracking	Undo Last Action	O(log n)	O(n)
Performance Analysis	Full Benchmark Suite	O(n log n)	O(n)

10.2 Variable Definitions

- n = number of active orders in the system
- K = number of kitchens (restaurants) in the system
- V = number of location nodes in the city graph ($V \leq 20$)
- N = total number of orders with recorded history
- m = number of history entries per order

10.3 Overall System Scalability

The dominant bottleneck at scale is the Order Scheduling Engine, which performs O(n log n) operations during peak processing. At 10,000 simultaneous orders, this remains well within practical real-time constraints (approximately 25ms). All other modules either operate in constant time (hash lookup, road blocking) or have complexity bounded by small constants ($V \leq 20$ for Dijkstra, K for kitchen management).

11. Ethical & Professional Considerations

11.1 Customer Data Privacy

The system stores customer names, order history, delivery addresses (as graph nodes), and payment-category information. The following practices ensure responsible data handling:

- Data Minimization: Only attributes strictly necessary for dispatch and optimization are stored. No financial data, contact numbers, or authentication credentials are handled within this system.
- Persistence Scope: CSV files are stored locally and are not transmitted over any network. In a production deployment, these files would be replaced by an encrypted database with access controls.
- Anonymization for Benchmarking: The Performance Analysis module uses synthetically generated order data with no real customer identifiers.

11.2 Algorithmic Fairness

The priority system (Normal / Premium / VIP) reflects customer service-level agreements established at subscription time. To prevent starvation of lower-priority orders:

- The deadline-based tie-breaking rule ensures Normal-priority orders with imminent deadlines are not indefinitely deferred by Premium orders with distant deadlines.
- A production system would additionally implement aging — incrementally raising an order's effective priority the longer it waits — to eliminate starvation entirely.

11.3 Rider Welfare

The load balancing formula normalizes workload to prevent systematic overloading of specific riders. The system tracks total deliveries per rider, enabling fair performance review. Rider availability flags allow riders to pause without losing their queue position.

11.4 System Reliability

Auto-save triggers after every mutating operation ensure that no data is lost in the event of unexpected termination. The undo mechanism provides an audit trail of corrective actions taken by operators.

12. Scalability & Future Enhancements

12.1 Current Scalability Limits

Component	Current Limit	Bottleneck Cause
City Graph	20 nodes	Fixed MAX_NODES array; redesign as dynamic adjacency list to scale
Hash Table	101 buckets	Fixed bucket count; load factor rises with many orders — needs dynamic rehashing
Array<T>	~2GB (heap memory)	Doubles capacity on overflow — practical limit is system RAM
CSV Persistence	~10,000 rows efficiently	Linear file scan on load; replace with indexed binary format at scale

12.2 Recommended Future Enhancements

- Replace adjacency matrix with a dynamic adjacency list to support hundreds of city locations.
- Implement dynamic hash table rehashing to maintain O(1) average performance as the order database grows.
- Add multi-threading for concurrent order processing (one thread per kitchen queue).
- Implement aging in the scheduling heap to eliminate starvation of low-priority orders.
- Replace CSV persistence with a binary-indexed file format or SQLite for faster load times.
- Introduce a geographic coordinate system (GPS) for Euclidean heuristic-based A* routing.
- Add a real-time dashboard visualization layer (web socket feed from the CLI engine).

13. Conclusion

This project demonstrates the design and implementation of a complete, multi-module Dispatch Optimization Engine for a food delivery platform using fundamental data structures and algorithms. Each module was built from first principles in C++ without reliance on the Standard Template Library, fulfilling the core academic objective of the course.

The project illustrates how the right choice of data structure — Max-Heap for priority scheduling, Linked-List queues for kitchen batching, Min-Heap for rider selection, Adjacency Matrix with Dijkstra for routing, Hash Table for search, and Stack for undo — directly determines system performance and correctness.

All design choices are backed by theoretical complexity analysis and validated through an experimental benchmarking module. The system handles dynamic insertions, cancellations, rerouting, and workload rebalancing in real time, confirming that algorithmic design, not raw hardware, is the foundation of scalable software systems.

End of Documentation

FoodExpress Dispatch Optimization Engine — CSC-200L OEL — UET Lahore, New Campus